



Servidores LSP

OpenCode se integra con sus servidores LSP.

OpenCode se integra con su protocolo de servidor de idiomas (LSP) para ayudar a LLM a interactuar con su código base. Utiliza diagnósticos para proporcionar retroalimentación al LLM.

Incorporados

OpenCode viene con varios servidores LSP integrados para idiomas populares:

SERVIDOR LSP	EXTENSIONES	REQUISITOS
astro	.astro	Autoinstalaciones para proyectos Astro
bash	.sh, .bash, .zsh, .ksh	Autoinstala el servidor en lenguaje bash
clangd	.c, .cpp, .cc, .cxx, .c++, .h, .hpp, .hh, .hxx, .h++	Instalaciones automáticas para proyectos C/C++
csharp	.cs	` <b>.NET SDK</b> ` instalado
clojure-lsp	.clj, .cljs, .cljs, .edn	Comando ` <b>clojure-lsp</b> ` disponible
dart	.dart	Comando ` <b>dart</b> ` disponible
deno	.ts, .tsx, .js, .jsx, .mjs	Comando ` <b>deno</b> ` disponible (detecta automáticamente deno.json/deno.jsonc)
elixir-ls	.ex, .exs	Comando ` <b>elixir</b> ` disponible
eslint	.ts, .tsx, .js, .jsx, .mjs, .cjs, .mts, .cts,	` <b>eslint</b> ` dependencia en proyecto

SERVIDOR LSP	EXTENSIONES	REQUISITOS
	.vue	
fsharp	.fs, .fsi, .fsx, .fsscript	` <b>.NET SDK</b> ` instalado
gleam	.gleam	Comando ` <b>gleam</b> ` disponible
gopls	.go	Comando ` <b>go</b> ` disponible
hls	.hs, .lhs	Comando ` <b>haskell-language-server-wrapper</b> ` disponible
jdts	.java	` <b>Java SDK (version 21+)</b> ` instalado
julia-ls	.jl	` <b>julia</b> ` y ` <b>LanguageServer.jl</b> ` instalados
kotlin-ls	.kt, .kts	Autoinstalaciones para proyectos Kotlin
lua-ls	.lua	Autoinstalaciones para proyectos Lua
nixd	.nix	Comando ` <b>nixd</b> ` disponible
ocaml-lsp	.ml, .mli	Comando ` <b>ocaml lsp</b> ` disponible
oxlint	.ts, .tsx, .js, .jsx, .mjs, .cjs, .mts, .cts, .vue, .astro, .svelte	` <b>oxlint</b> ` dependencia en proyecto
php intelephense	.php	Autoinstalaciones para proyectos PHP
prisma	.prisma	Comando ` <b>prisma</b> ` disponible
pyright	.py, .pyi	Dependencia ` <b>pyright</b> ` instalada
ruby-lsp (rubocop)	.rb, .rake, .gemspec, .ru	Comandos ` <b>ruby</b> ` y ` <b>gem</b> ` disponibles
rust	.rs	Comando ` <b>rust-analyzer</b> ` disponible
sourcekit-lsp	.swift, .objc, .objcpp	` <b>swift</b> ` instalado (` <b>xcode</b> ` en macOS)

SERVIDOR LSP	EXTENSIONES	REQUISITOS
svelte	.svelte	Autoinstalaciones para proyectos Svelte
terraform	.tf, .tfvars	Instalaciones automáticas desde versiones GitHub
tinymist	.typ, .typc	Instalaciones automáticas desde versiones GitHub
typescript	.ts, .tsx, .js, .jsx, .mjs, .cjs, .mts, .cts	` <b>typescript</b> ` dependencia en proyecto
vue	.vue	Autoinstalaciones para proyectos Vue
yaml-ls	.yaml, .yml	Autoinstala Red Hat yaml-language-server
zls	.zig, .zon	Comando ` <b>zig</b> ` disponible

Los servidores LSP se habilitan automáticamente cuando se detecta una de las extensiones de archivo anteriores y se cumplen los requisitos.

NOTA

Puede deshabilitar las descargas automáticas del servidor LSP configurando la variable de entorno `**OPENCODE\_DISABLE\_LSP\_DOWNLOAD**` en `**true**`.

Cómo funciona

Cuando opencode abre un archivo:

Comprueba la extensión del archivo con todos los servidores LSP habilitados.

Inicia el servidor LSP apropiado si aún no se está ejecutando.

Configuración

Puede personalizar los servidores LSP a través de la sección ``lsp`` en su configuración `opencode`.

`opencode.json`

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": {}
}
```

Cada servidor LSP admite lo siguiente:

PROPIEDAD	TIPO	DESCRIPCIÓN
<code>`disabled`</code>	booleano	Establezca esto en <code>`true`</code> para deshabilitar el servidor LSP
<code>`command`</code>	cadena[]	El comando para iniciar el servidor LSP
<code>`extensions`</code>	cadena[]	Extensiones de archivo que este servidor LSP debería manejar
<code>`env`</code>	objeto	Variables de entorno para configurar al iniciar el servidor
<code>`initialization`</code>	objeto	Opciones de inicialización para enviar al servidor LSP

Veamos algunos ejemplos.

Variables de entorno

Utilice la propiedad ``env`` para establecer variables de entorno al iniciar el servidor LSP:

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": {
    "rust": {
      "env": {
        "RUST_LOG": "debug"
      }
    }
  }
}
```

Opciones de inicialización

Utilice la propiedad ``initialization`` para pasar opciones de inicialización al servidor LSP. Estas son configuraciones específicas del servidor enviadas durante la solicitud LSP ``initialize``:

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": {
    "typescript": {
      "initialization": {
        "preferences": {
          "importModuleSpecifierPreference": "relative"
        }
      }
    }
  }
}
```

❗ NOTA

Las opciones de inicialización varían según el servidor LSP. Consulte la documentación de su servidor LSP para conocer las opciones disponibles.

Deshabilitar servidores LSP

Para deshabilitar todos los servidores LSP globalmente, configure ``lsp`` en ``false``:

opencode.json
<pre>{ "\$schema": "https://opencode.ai/config.json", "lsp": false }</pre>

Para deshabilitar un servidor LSP específico, configure ``disabled`` en ``true``:

opencode.json
<pre>{ "\$schema": "https://opencode.ai/config.json", "lsp": { "typescript": { "disabled": true } } }</pre>

Servidores LSP personalizados

Puede agregar servidores LSP personalizados especificando el comando y las extensiones de archivo:

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": {
    "custom-lsp": {
      "command": ["custom-lsp-server", "--stdio"],
      "extensions": [".custom"]
    }
  }
}
```

Información adicional

PHPintelephense

PHPintelephense ofrece funciones premium a través de una clave de licencia. Puede proporcionar una clave de licencia colocando (únicamente) la clave en un archivo de texto en:

En macOS/Linux: ``$HOME/intelephense/license.txt``

En Windows: ``%USERPROFILE%/intelephense/license.txt``

El archivo debe contener sólo la clave de licencia sin contenido adicional.

 Edita esta página

© Anomaly

 Found a bug? Open an issue

Última actualización: 21 may 2026

 Join our Discord community

🚩 Español 